CSS Full Notes

Simple explanations, one topic at a time: from selectors and the box model to Flexbox, Grid, responsive design and animations

Getting Started

What is CSS?

CSS means Cascading Style Sheets. HTML builds the page. CSS makes it look good: colors, fonts, spacing and layout.

```
p { color: blue; }
```

CSS syntax

A rule has a selector (what to style) and a block of declarations. Each declaration is a property and a value, ending with a semicolon.

```
h1 {
  color: red;      /* property: value */
  font-size: 32px;
}
```

3 ways to add CSS

Inline: on one tag. Internal: in a style tag. External: a separate .css file. External is best.

```
<p style="color:red">
<style> p{color:red} </style>
<link rel="stylesheet" href="s.css">
```

Comments

Comments are notes for you. The browser ignores them.

```
/* this is a comment */
```

Selectors

Basic selectors

Element selects by tag. Class (.) selects many elements. ID (#) selects one. * selects everything.

```
p { }
.card { }
#logo { }
* { margin: 0; }
```

Group & combine

Use a comma to style several selectors at once. Combine a tag and a class with no space.

```
h1, h2, h3 { color: navy; }
p.big { font-size: 20px; }
```

Descendant & child

A space picks any element inside. > picks only direct children.

```
nav a { }      /* any a inside nav */
ul > li { }     /* direct children */
```

Attribute selectors

Select elements by their attributes.

```
input[type="text"] { }
a[href^="https"] { }
```

Pseudo-classes & Specificity

Pseudo-classes

They style an element in a special state, like when hovered or focused.

```
a:hover { color: red; }
input:focus { outline: 2px solid blue; }
li:first-child { }
```

Pseudo-elements

They style part of an element or add content. They use two colons.

```
p::first-letter { font-size: 2em; }
.btn::after { content: " >"; }
```

Sibling selectors

+ picks the next sibling. ~ picks all following siblings.

```
h2 + p { margin-top: 0; }
h2 ~ p { color: gray; }
```

Specificity

When rules clash, the more specific selector wins. Order: inline, then ID, then class, then element. Later rules win a tie.

```
p { color: red; }
.a { color: blue; }  /* wins */
#b { color: green; } /* wins most */
```

Colors & Backgrounds

Color formats

Colors can be names, HEX, RGB, or HSL. Add alpha for see-through colors.

```
color: red;
color: #ff0000;
color: rgb(255 0 0 / 50%);
color: hsl(0 100% 50%);
```

Background color & image

background-color fills the box. background-image puts a picture behind it.

```
background-color: #fff3a8;
background-image: url("bg.jpg");
```

Background controls

Decide how the image repeats, where it sits and how big it is.

```
background-repeat: no-repeat;
background-position: center;
background-size: cover;
```

Opacity

opacity makes the whole element see-through, from 0 to 1.

```
img { opacity: 0.5; }
```

Text & Fonts

Font basics

Choose the typeface, size, weight and style. Give backup fonts in case the first is missing.

```
font-family: "Arial", sans-serif;
font-size: 16px;
font-weight: bold;
font-style: italic;
```

Text styling

Align, decorate, space and change the case of text.

```
text-align: center;
text-decoration: underline;
text-transform: uppercase;
letter-spacing: 1px;
```

Line height & spacing

line-height sets the space between lines. 1.5 is easy to read.

```
line-height: 1.5;
word-spacing: 4px;
```

Web fonts

Load free fonts from Google Fonts or your own file with @font-face.

```
@font-face {
  font-family: "My";
  src: url("my.woff2");
}
```

The Box Model

What is the box model?

Every element is a box with four layers: content, padding, border and margin, from inside to outside.

```
content > padding > border > margin
```

Padding & margin

Padding is space inside the border. Margin is space outside. One value = all sides. Four values = top right bottom left.

```
padding: 10px 20px;
margin: 0 auto;  /* center a block */
```

Border

A border needs a width, a style and a color. Radius rounds the corners.

```
border: 2px solid black;
border-radius: 8px;
```

box-sizing

With border-box, padding and border stay inside the width you set. Use it on everything.

```
* { box-sizing: border-box; }
```

Units & Values

Absolute & relative units

px is a fixed pixel. % is a share of the parent. em follows the parent font size. rem follows the root font size.

```
width: 50%;
font-size: 1.2rem;
margin: 2em;
```

Viewport units

vw and vh are 1 percent of the screen width and height.

```
height: 100vh;
width: 50vw;
```

calc, min, max, clamp

Do math in CSS. clamp sets a smallest, ideal and biggest size.

```
width: calc(100% - 40px);
font-size: clamp(1rem, 4vw, 2rem);
```

Width & height

Set sizes, and limit them with min and max.

```
width: 300px;
max-width: 100%;
min-height: 200px;
```

Display & Position

display

block takes the full line. inline stays in the line. inline-block is in the line but accepts size. none hides the element.

```
display: block;
display: inline-block;
display: none;
```

position

static is normal. relative shifts from its own place. absolute sits inside the nearest positioned parent. fixed sticks to the screen. sticky sticks while scrolling.

```
position: absolute;
top: 10px;
right: 10px;
```

z-index

Controls which element is on top when they overlap. It works on positioned elements.

```
z-index: 10;
```

overflow & visibility

overflow decides what happens when content is too big. visibility hides but keeps the space.

```
overflow: hidden;
overflow: auto;
visibility: hidden;
```

Flexbox: Basics

Turn on flex

Set display: flex on the parent. Its children become flex items in a row.

```
.box { display: flex; }
```

flex-direction

Choose row (left to right) or column (top to bottom).

```
flex-direction: row;
flex-direction: column;
```

justify-content

Spreads items along the main direction.

```
justify-content: center;
justify-content: space-between;
```

align-items

Aligns items across the other direction. Center both to center anything.

```
align-items: center;
/* perfect centering */
justify-content: center;
```

Flexbox: More

flex-wrap & gap

wrap lets items drop to the next line. gap adds space between items.

```
flex-wrap: wrap;
gap: 16px;
```

flex-grow & shrink

On the items: grow lets them take extra space. shrink lets them get smaller.

```
.item { flex: 1; }
/* grow equally */
```

order & align-self

order changes the position. align-self moves one item alone.

```
.item { order: -1; }
.item { align-self: flex-end; }
```

When to use flex

Flexbox is best for one direction at a time: menus, cards in a row, centering.

```
nav { display: flex; gap: 12px; }
```

Grid Layout

Turn on grid

display: grid makes rows and columns. Set the columns with grid-template-columns.

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 12px;
}
```

fr unit & repeat

fr means a share of free space. repeat saves typing.

```
grid-template-columns: repeat(3, 1fr);
```

Responsive grid

auto-fit with minmax makes a grid that adjusts without media queries.

```
grid-template-columns:
  repeat(auto-fit, minmax(200px, 1fr));
```

Place items & areas

Make an item span columns, or name the layout areas.

```
.wide { grid-column: span 2; }
grid-template-areas: "h h" "s m";
```

Responsive Design

Why responsive?

People use phones, tablets and computers. One page should look good on all.

```
<meta name="viewport"
 content="width=device-width, initial-scale=1">
```

Media queries

Apply CSS only when a condition is true, like a screen width.

```
@media (max-width: 600px) {
  .menu { display: none; }
}
```

Mobile first

Write CSS for phones first. Then add changes for bigger screens with min-width.

```
@media (min-width: 768px) {
  .grid { grid-template-columns: 1fr 1fr; }
}
```

Flexible images

Make images never go wider than their box.

```
img { max-width: 100%; height: auto; }
img { object-fit: cover; }
```

Styling Links, Lists, Tables & Forms

Links

a has four states: link, visited, hover and active. Remove the underline with text-decoration.

```
a { text-decoration: none; }
a:hover { color: red; }
```

Lists

Change or remove bullets, and set the marker position.

```
ul { list-style: none; padding: 0; }
ol { list-style-type: upper-roman; }
```

Tables

Merge borders, add padding, and stripe rows.

```
table { border-collapse: collapse; }
td { padding: 8px; }
tr:nth-child(even) { background: #eee; }
```

Forms

Style inputs, buttons and the focus state. Do not remove focus outlines without a replacement.

```
input { padding: 8px; border: 1px solid #ccc; }
button:hover { background: navy; }
```

Shadows, Gradients & Effects

box-shadow

Adds a shadow around a box: x, y, blur and color.

```
box-shadow: 2px 4px 8px rgba(0,0,0,.3);
```

text-shadow

Adds a shadow to text.

```
text-shadow: 1px 1px 2px gray;
```

Gradients

A smooth blend of colors, used as a background.

```
background: linear-gradient(to right, #f9d, #9df);
background: radial-gradient(circle, #fff, #ccc);
```

Filters

Blur, brighten or gray out elements and images.

```
filter: blur(2px);
filter: grayscale(100%);
```

Transitions & Animations

transition

Makes a change happen smoothly instead of instantly. Great for hover effects.

```
.btn {
  transition: background 0.3s ease;
}
.btn:hover { background: red; }
```

transform

Move, rotate, scale or tilt an element without changing the layout.

```
transform: translateX(20px);
transform: rotate(45deg);
transform: scale(1.2);
```

@keyframes

Define the steps of an animation by name.

```
@keyframes spin {
  from { transform: rotate(0); }
  to { transform: rotate(360deg); }
}
```

animation

Apply the keyframes with a duration and repeat count.

```
.loader {
  animation: spin 1s linear infinite;
}
```

Modern & Advanced CSS

CSS variables

Store a value once and reuse it. Change it in one place to update everywhere.

```
:root { --main: #09f; }
a { color: var(--main); }
```

Inheritance & cascade

Some properties, like color and font, pass down to children. The cascade decides which rule wins.

```
body { color: #333; }
/* children inherit it */
```

Newer selectors

These make CSS shorter and smarter.

```
:is(h1, h2) { margin: 0; }
card:has(img) { padding: 0; }
```

Container queries & nesting

Style a component by the size of its container. Nesting lets you write rules inside rules.

```
.card {
  &:hover { color: red; }
}
@container (min-width: 400px) { }
```

Best Practices & Key Takeaways

Write clean CSS

Use clear class names. Keep related rules together. Avoid !important and deep selectors.

```
.card-title { }   /* clear name */
```

Reset and organize

Start with a small reset. Split big files into parts: base, layout, components.

```
* { margin: 0; box-sizing: border-box; }
```

Use DevTools

Press F12 and click Inspect. You can see and test CSS live in the browser.

```
F12 > Elements > Styles
```

Learning path

1. Selectors, colors, text. 2. Box model and units. 3. Display and position. 4. Flexbox and Grid. 5. Responsive design. 6. Animations. 7. Build real pages.

Try each example in a style.css file linked to your HTML page.